Assignment 2, Univariate Simulation

B: Importance Sampling

Carl-Emil Krarup Jensen

University of Copenhagen
Department of Mathematical Sciences

October 28, 2026

Agenda

- Theoretical background and implementation
- Numerical comparisons
- Benchmarking and profiling
- Discussion of choices and potential improvements

Theoretical background

Let $X_1, X_2 \ldots, X_n \overset{i.i.d}{\sim} \mathcal{U}((-1.9, 2))$, and define

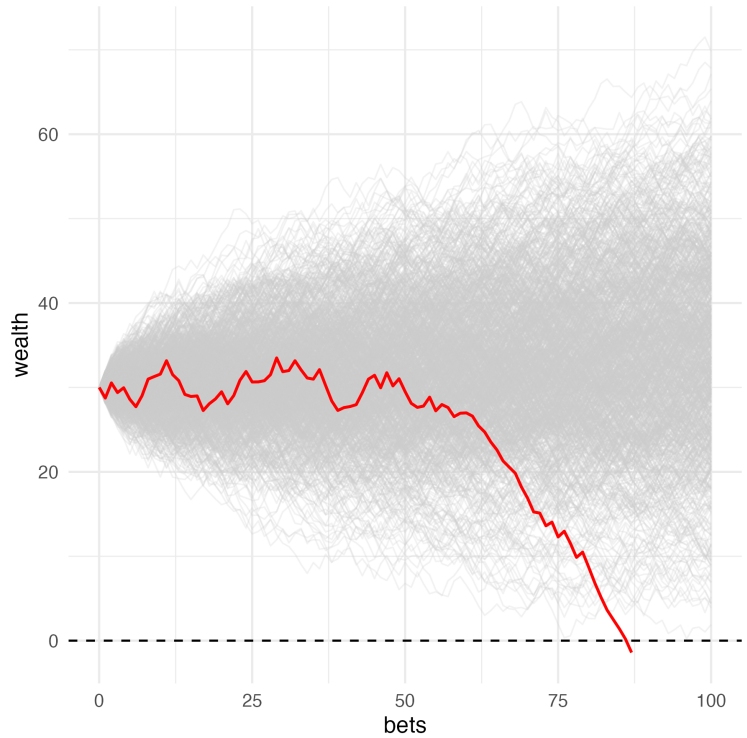
$$S_n = 30 + \sum_{k=1}^{n} X_k.$$

The goal is to estimate the probability $p(n)$, that S_k reaches 0 or below, before or at n;

$$p(n) = \mathbb{P}(\exists k \leq n \mid S_k \leq 0).$$

We will be focusing on the case where $n = 100$.

In different terminology $p(n)$ can be expressed as gamblers ruin probabilities: The probability that a gambler accumulating wealth through bets $X_1, X_2 \ldots, X_n$, loose all wealth before or at bet n.



Simulated paths of S_{100} until one reaches 0 or below.

Theoretical background and implementation

We use Monte Carlo integration and Importance Sampling to estimate $p(100)$. Let $A_n = \{\exists k \leq n \mid S_k \leq 0\}$ and let f denote the density of $X = (X_1, X_2, \ldots, X_n)$. Let $g_{\theta,n}(y) = (\varphi(\theta))^{-n} \exp\left(\theta \sum_{k=1}^{n} y_k\right)$, with $\varphi(\theta) = \int_{-1.9}^{2} e^{\theta z} dz$, and let $Y = (Y_1, Y_2, \ldots, Y_n)$, with $Y_k \overset{i.i.d}{\sim} g_\theta$.

The the standard Monte Carlo estimator is

$$\hat{p}_{MC}(100) = \frac{1}{N} \sum_{i=1}^{N} \mathbf{1}_{A_{100}}.$$

The Importance Sampling estimator is

$$\hat{p}_{IS}(100) = \frac{1}{N} \sum_{j=1}^{N} \mathbf{1}_{A_{100}} \frac{f(Y^{(j)})}{g_{\theta,100}(Y^{(j)})}.$$

Project structure

```
assignment_2
├── project_B
│   ├── experiment.R
│   ├── importance_sampling.R
│   ├── model.R
│   ├── simulation.R
│   └── theory.R
├── report
└── tests/testthat
    ├── test-importance_sampling.R
    ├── test-model.R
    └── test-simulation.R
```

Theoretical background and implementation

Why is importance sampling well suited for this problem?
The indicator $\mathbf{1}_{A_{100}}$ is Bernoulli, with a very small probability parameter p. The relative standard error of $\hat{p}_{MC}$ is

$$RSE\left(\hat{p}_{MC}\right) = \sqrt{\frac{1-p}{Np}}.$$

For a fixed N and a small p the error is, large relative to p. Let $L_{\theta,n}(Y) = \frac{f(Y)}{g_{\theta,n}(Y)}$, changing the background measure $\mathbb{P}$ to $\mathbb{P}_g$, $\mathbb{P} \ll \mathbb{P}_g$ we have by Radon Nikodym

$$\mathbb{E}\left(\mathbf{1}_{A_{100}}\right) = \mathbb{E}_{\mathbb{P}_g}\left(\mathbf{1}_{A_{100}} L_{\theta,n}(Y)\right) = p.$$

Then

$$\mathrm{Var}_{\mathbb{P}_g}\left(\mathbf{1}_{A_{100}} L_{\theta,100}(Y)\right) = \mathbb{E}_{\mathbb{P}_g}\left(\mathbf{1}_{A_{100}} L^2_{\theta,100}(Y)\right) - p^2.$$

So

$$\mathrm{Var}_{\mathbb{P}_g}\left(\hat{p}_{IS}\right) = \frac{1}{N}\mathrm{Var}_{\mathbb{P}_g}\left(\mathbf{1}_{A_{100}} L_{\theta,100}(Y)\right)$$

And consequently

$$RSE\left(\hat{p}_{IS}\right) = \sqrt{\frac{1}{N}\left(\frac{\mathbb{E}_{\mathbb{P}_g}\left(\mathbf{1}_{A_{100}} L^2_{\theta,100}(Y)\right)}{p^2} - 1\right)}.$$

Theoretical background and implementation

Importance sampling reduces relative error when

$$\frac{\mathbb{E}_{\mathbb{P}_g}\left(\mathbf{1}_{A_{100}} L^2_{\theta,100}(Y)\right)}{p^2} < \frac{1}{p} \iff \mathbb{E}_{\mathbb{P}_g}\left(\mathbf{1}_{A_{100}} L^2_{\theta,100}(Y)\right) < p$$

Let $q_\theta = \mathbb{P}_g(A_n)$, by the law of total variance, we have

$$\mathbb{E}\left(L^2_{\theta,100}(Y) \mid A_{100}\right) = q_{\theta,100}\left(\mathrm{Var}_{\mathbb{P}_g}\left(L_{\theta,100}(Y) \mid A_{100}\right) + \frac{p^2}{q^2_{\theta,100}}\right).$$

The relative standard error can thus be decomposed as

$$\mathrm{RSE}_{IS} = \sqrt{\frac{1}{N}\left(\frac{1-q_{\theta,100}}{q_{\theta,100}} + \frac{q_{\theta,100}}{p^2}\,\mathrm{Var}_{\mathbb{P}_g}(L_{\theta,100} \mid A_{100})\right)}.$$

- θ more negative: ruin becomes more frequent.
- However making theta too big numerically will make the second term more sensitive to increment variations
- Optimal θ: balances the two effects.

Theoretical background and implementation

The estimators are wrapped in a function in `importance_sampling.R` taking the argument `stopped`, `full`, `crude` to specify the estimator. The argument `n` is the number of trajectories N; `horizon` is the number of bets.

```
estimate_ruin <- function(
    n, theta = 0, method = c("stopped", "full", "crude"),
    initial_capital = 30, horizon = 100,
    lower = -1.9, upper = 2, diagnostics = FALSE) {

  method <- match.arg(method)

  # At theta = 0, the tilted distribution is uniform.
  if (method == "crude") theta <- 0

  # One vector entry for each independent trajectory.
  capital <- rep(initial_capital, n)
  times <- rep(Inf, n)  # No ruin observed yet.
  steps <- integer(n)  # Number of increments generated.
```

Theoretical background and implementation

At each time step, update all active trajectories:

$$S_k^{(j)} = S_{k-1}^{(j)} + Y_k^{(j)}, \qquad Y_k^{(i)} \sim g_\theta.$$

```
for (k in seq_len(horizon)) {

  # Stopped IS updates only paths without recorded ruin.
  # Full IS and crude MC continue updating every path.
  active <- if (method == "stopped") {
    which(is.infinite(times))
  } else {seq_len(n)}

  # No further simulation is needed if all paths stopped.
  if (length(active) == 0) break
  capital[active] <- capital[active] +
    r_tilted(length(active), theta, lower, upper)

  steps[active] <- k
```

Each active path receives one independent increment.

Theoretical background and implementation

For trajectory k, the first ruin time is $\tau_i = \inf\{k \geq 1 : S_k^{(i)} \leq 0\}$.

```
    # A new ruin requires:
    # 1. no previously recorded crossing;
    # 2. current capital at or below zero.
    newly_ruined <- active[
      is.infinite(times[active]) & capital[active] <= 0
    ]

    # Record the first crossing only.
    times[newly_ruined] <- k
  }  # End of the time-step loop.

  # TRUE exactly when ruin occurred within the horizon.
  ruined <- is.finite(times)
```

Passing the argument `full`, then a ruined path continues, but its first ruin time is retained.

Passing the argument `stopped`, then a ruined path is excluded from subsequent updates.

Theoretical background and implementation

We use that $\log \frac{f(Y)}{g(Y)} = n\frac{\varphi(\theta)}{3.9} - \theta \sum_{k=1}^{n} y_k$, in `log_likelihood_ratio()`, which we call:

```
# Every non-ruined trajectory contributes zero.
contributions <- numeric(n)

# On ruined trajectories, calculate L_m = exp(log L_m).
contributions[ruined] <- exp(log_likelihood_ratio(
  capital[ruined] - initial_capital,  # Sum of increments.
  steps[ruined],                      # Number of increments.
  theta, lower, upper
))
```

$$Z_j = \begin{cases} \mathbf{1}_{\{\tau_j \leq 100\}} L_{100}^{(j)}, & \text{full IS,} \\ \mathbf{1}_{\{\tau_j \leq 100\}} L_{\min(\tau_i, 100)}^{(j)}, & \text{stopped IS.} \end{cases}$$

Passing `crude` sets, $\theta = 0$ gives $L_m = 1$, so each contribution is simply zero or one collapsing to regular Monte Carlo.

Theoretical background and implementation

All three methods estimate the probability by

$$\hat{p} = \frac{1}{N} \sum_{j=1}^{N} Z_j.$$

```
# Summarize all contributions, including the zeros.
result <- list(
  summary = summarize_contributions(
    contributions, ruined, steps
  ),
  method = method,
  theta = theta
)
```

Inside `summarize_contributions()`, the calculations are:

$$\hat{p} = \texttt{mean(contributions)}, \qquad s_Z^2 = \texttt{var(contributions)},$$

$$\widehat{\mathrm{Var}}(\hat{p}) = \frac{s_Z^2}{N}, \qquad \widehat{\mathrm{SE}}(\hat{p}) = \sqrt{\frac{s_Z^2}{N}}.$$

Testing the implementation

```
test_that("independent unbiased estimators are consistent within MC uncertainty", {
  set.seed(5301)
  results <- lapply(c("crude", "full", "stopped"), function(method) {
    estimate_ruin(80000, -0.28, method, diagnostics = TRUE)
  })
  for (i in 1:2) for (j in (i + 1):3) {
    a <- results[[i]]$summary
    b <- results[[j]]$summary
    expect_lt(abs(a$estimate - b$estimate), 5 *
    sqrt(a$standard_error^2 + b$standard_error^2))
  }
  stopped <- results[[3]]
  expect_equal(stopped$steps, pmin(stopped$ruin_times, 100))
  expect_equal(stopped$summary$estimator_variance,
               var(stopped$contributions) / length(stopped$contributions))
  expect_equal(results[[1]]$contributions,
  as.numeric(is.finite(results[[1]]$ruin_times)))
  expect_true(all(results[[2]]$steps == 100))
})
```

Theoretical background and implementation

From a numerical perspective the objective becomes to find θ^*_{num} satisfying

$$\theta^*_{num} = \underset{\theta \in \Theta}{\operatorname{argmin}}\, s_Z^2$$

In practice we divide Θ into a fine grid and search for θ^* via simulations in each grid point.

From a computational perspective the optimization is different. Given a budget B and a cost of simulating a path $c(\theta)$ we may simulate $N = \frac{B}{c(\theta)}$ repetitions. The goal is then to find the θ^*_{comp} satisfying

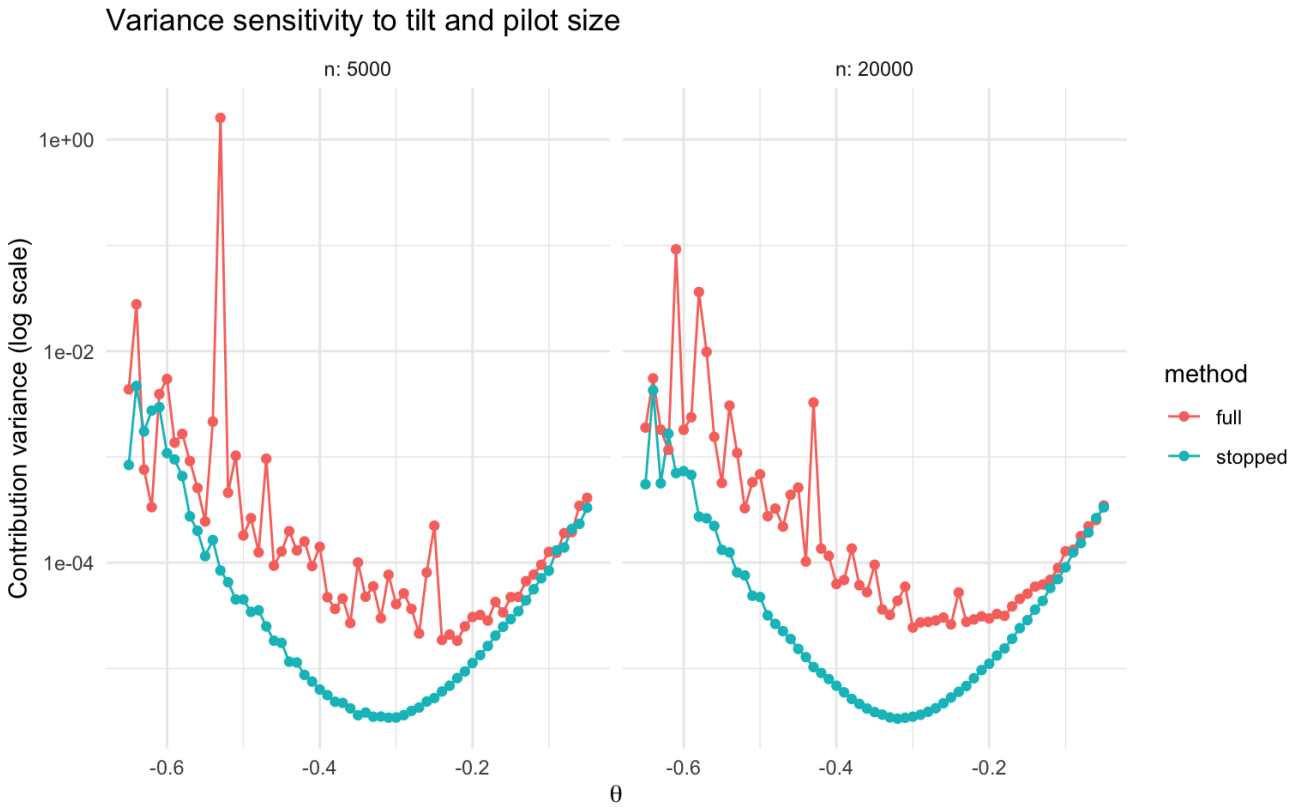
$$\theta^*_{comp} = \underset{\theta \in \Theta}{\operatorname{argmin}}\, s_Z^2 c(\theta).$$

We will start by searching for θ^*_{num}.

Numerical comparisons

Finding θ^*_{num} by searching Θ

```
pilot <- variance_experiment(theta_grid, c(5000, 20000), seed = 2101)
ggplot(pilot, aes(theta, contribution_variance, color = method)) + ...
```



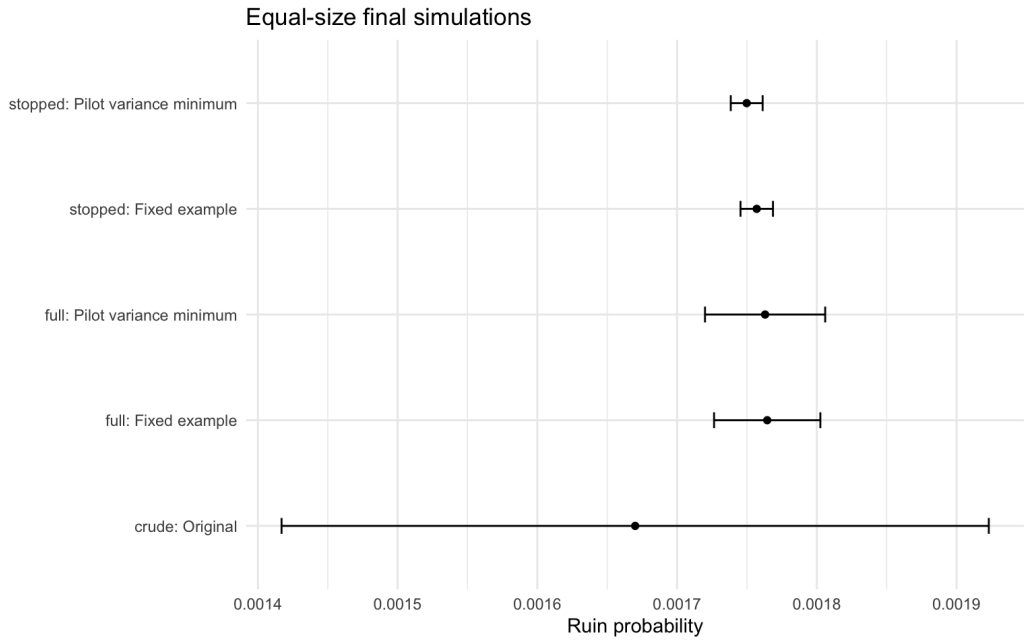
Empirical variance for the full and stopped method plotted against θ values.

Numerical comparisons

We can improve the estimate of p a lot buy setting optimal $\theta = \theta^*_{num}$.
Confidence intervals are computed by $\hat{\text{se}} = \hat{p} \pm \texttt{qnorm(0.975)} s_Z$

Method	θ	Estimate	$\hat{\text{se}}$
Crude	0.00	0.001670	0.000129
Full	-0.3	0.001763	0.000022
Stopped	-0.32	0.001750	0.000006

- The importance sampling estimators don't even cover the crude MC estimator in their 95% confidence intervals
- The stopped importance sampling estimator has an even tighter confidence interval, illustrating that discarding random weight after ruin improves the estimate.



Estimates of p with 95% Monte Carlo confidence intervals

Let $T = \min\{100, \tau\}$, then

$$\text{Var}_{\mathbb{P}_g}(L_{\theta,100}) = \text{Var}(L_{\theta,T}) + \underbrace{\mathbb{E}_{\mathbb{P}_g}(\text{Var}(L_{\theta,100} \mid \mathscr{F}_T))}_{\geq 0}$$

Computational complexity

Importance sample draws are more expensive than regular full Monte Carlo draws

```
X <- runif(100, min = -1.9, max = 2)
U <- runif(100)
Y <- -1.9 + log1p(U * expm1(3.9 * theta)) / theta
```

Where we use inverse transformation sampling to sample Y.
Asymptotically both have complexity $O(Nn)$, since we perform N MC repetitions of $n = 100$ draws.

Let $c_{IS}(\theta)$ denote the cost of sampling Y and c_{MC} the cost of sampling X. We can compare the efficiency gain G through

$$G = \frac{Var(\hat{p}_{MC})c_{MC}(\theta)}{Var(\hat{p}_{IS})c_{IS}(\theta)}$$

G determines at what fraction of a budget B Importance Sampling takes to reach the same variance as the MC estimator.

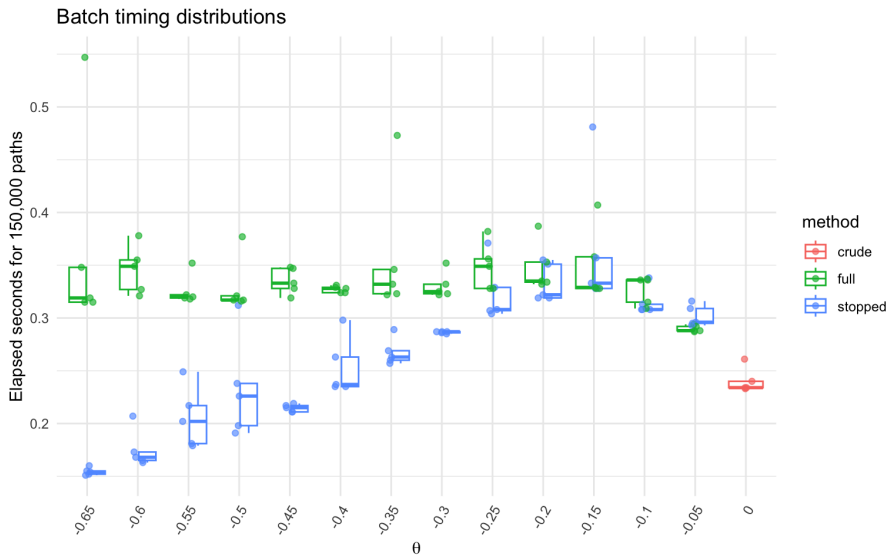
Computational complexity

The stopped estimator may improve the asymptotic complexity:

Recall $\tau = \inf\{k \geq 1 : S_k \leq 0\}$. Let $C_{\texttt{stop}}(\theta)$ be the cost of the stopped importance sampling algorithm, then

$$C_{\texttt{stop}}(\theta) \propto N\mathbb{E}_{\mathbb{P}_g}(\min(\tau, n))$$

Decreasing θ will decrease $\mathbb{E}_{\mathbb{P}_g}(\min(\tau, n))$, and thus, the complexity.



Runtime comparisons for different θ values

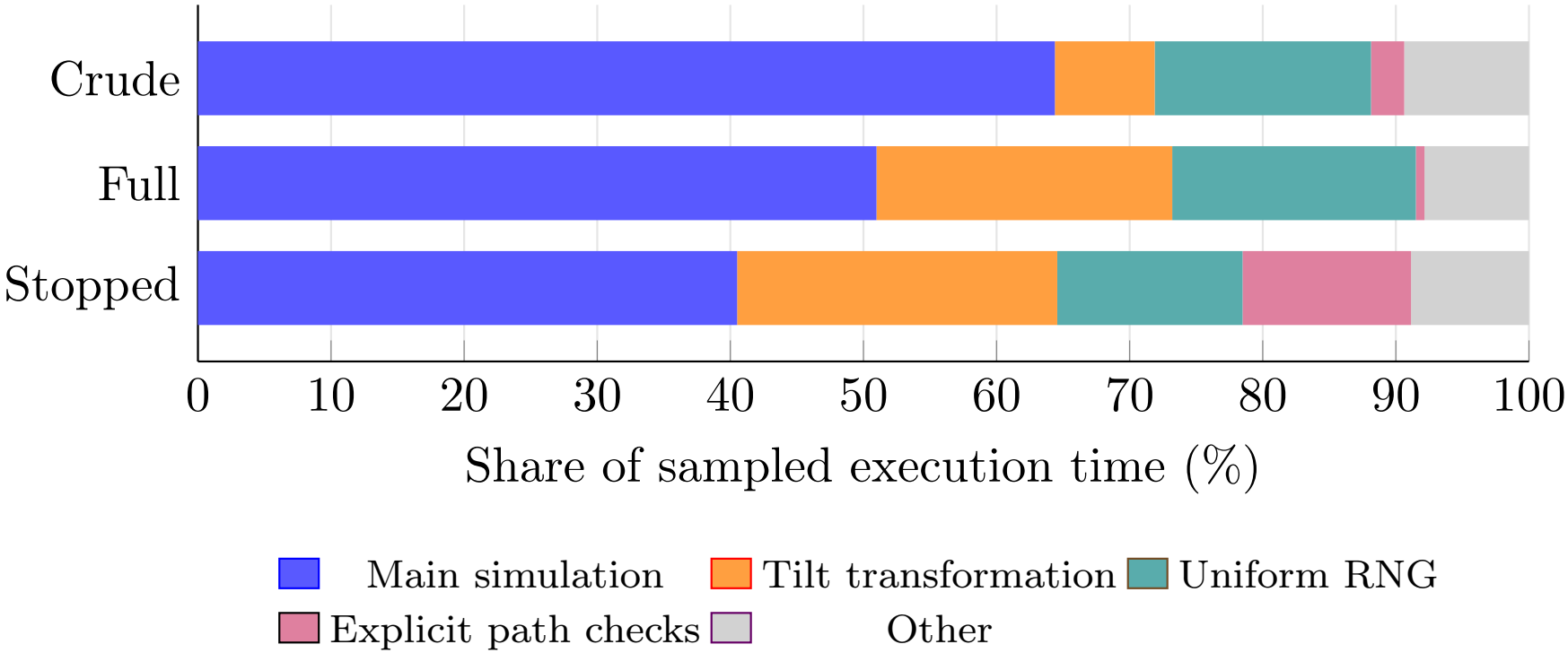
The worst case computational cost is still $O(Nn)$, what changes with θ is the expected computational cost.

Benchmarking and profiling

To identify bottlenecks in the code we profile it:

```
profile_method <- function(method, theta, n = 20000,
seconds = 2, file = tempfile(fileext = ".out"), seed = 4101) {
  set.seed(seed)
  start <- proc.time()[["elapsed"]]
  Rprof(file, interval = 0.01)
  on.exit(Rprof(NULL), add = TRUE)
  repeat {
    estimate_ruin(n, theta, method)
    if (proc.time()[["elapsed"]] - start >= seconds) break
  }
  Rprof(NULL)
  summaryRprof(file)$by.self
}
```

Benchmarking and profiling



- Simulation updates and increment generation dominate.
- Stopping introduces additional active-path checks.

The most expensive functions are `estimate_ruin` and `r_tilted`

Benchmarking and profiling

Examining the specific lines in the code causing the most execution time we see

Operation	location	Crude	Full IS	Stopped IS
Detect new ruin	`importance_sampling.R` line 42	32.8%	28.4%	29.6%
Update capital	`importance_sampling.R` line 40	32.6%	24.7%	17.0%
Generate uniforms	`model.R` line 45	20.7%	14.1%	13.3%
Inverse transformation	`model.R` line 50	—	26.0%	22.7%

Potential improvements suggested by profiling

- Stopped paths in `active` have not ruined: checking this again is redundant.
- Maintain surviving indices instead of scanning the entire ruin-time vector at every step.

Benchmarking and profiling

Original implementation

```
for (k in seq_len(horizon)) {

  # Rebuild survivors from all ruin times.
  active <- which(is.infinite(times))

  if (length(active) == 0) break

  capital[active] <- capital[active] +
    r_tilted(length(active),
             theta, lower, upper)
  steps[active] <- k

  # Check again that ruin was not recorded.
  newly_ruined <- active[
    is.infinite(times[active]) &
    capital[active] <= 0
  ]

  times[newly_ruined] <- k
}
```

New implementation

```
# Start with every trajectory active.
active <- seq_len(n)

for (k in seq_len(horizon)) {
  if (length(active) == 0) break

  capital[active] <- capital[active] +
    r_tilted(length(active),
             theta, lower, upper)
  steps[active] <- k

  # Active paths have not previously ruined.
  hit <- capital[active] <= 0
  times[active[hit]] <- k

  # Retain only the surviving indices.
  active <- active[!hit]
}
```

- Avoid scanning all ruin times at every step.
- Remove the redundant previous-ruin check.
- Preserve survivor order and identical simulated results.

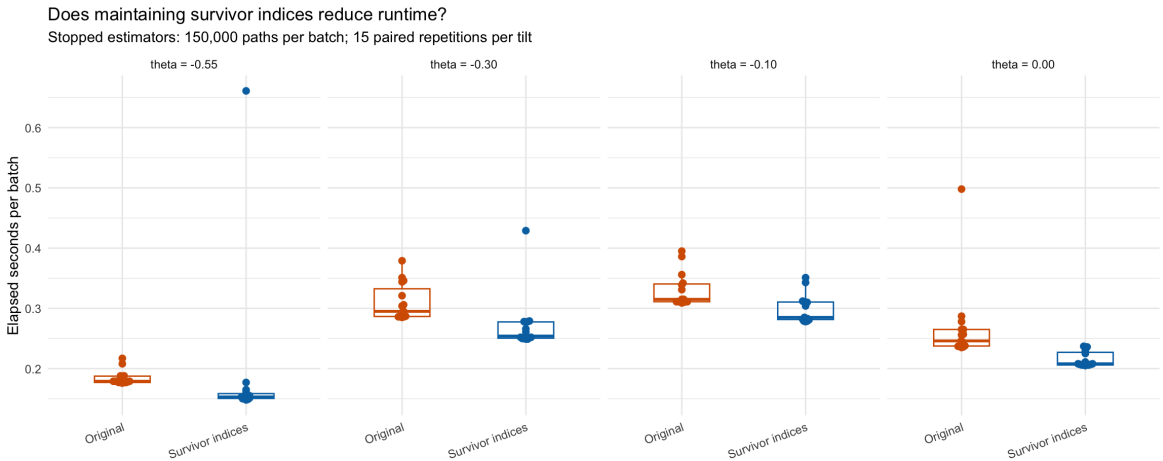
Benchmarking and profiling

We test and benchmark the new implementation.

We use $N = 150,000$ and perform 15 repetitions at each of four $\theta \in \{-0.6, -0.3, -0.05, 0, 0.3\}$

Runtime comparison

θ	Original (s)	Improved (s)	Saved (%)
-0.55	0.179	0.153	14.5
-0.30	0.295	0.254	13.1
-0.10	0.315	0.285	10.0
0.00	0.246	0.208	13.1




```
  test_that("survivor indexing
  exactly reproduces
  stopped simulations", {

  for (theta in
  c(-0.6, -0.3,-0.05, 0, 0.3)) {

  set.seed(6101)
  original <- estimate_ruin(
      2000, theta, "stopped",
      diagnostics = TRUE)

  set.seed(6101)
  improved <- estimate_ruin_survivors(
  2000, theta,
  diagnostics = TRUE)

  expect_identical(improved, original)
}})
```

Benchmarking and profiling

The updated lines improve runtime across multiple configurations, saving between 10% and 14% of median runtime in our experiments

- Through iterative benchmarking and profiling we can reduce runtime.
- We are however limited by the asymptotic computational complexity if we don't discover algorithmic optimization options
- Before doing so, we can maximize the computational efficiency

We will be using the improved stopped algorithm to compute estimates of

$$G = \frac{\text{Var}(\hat{p}_{IS}(\theta_i))c_{IS}(\theta_i)}{\text{Var}(\hat{p}_{IS}(\theta_j)c_{IS}(\theta_j)}, \qquad i \neq j$$

Benchmarking and profiling

We examine a grid in the interval $[-0.45, -0.15]$, with equidistant points in steps of 0.025, with three 50,000-path batches per θ.

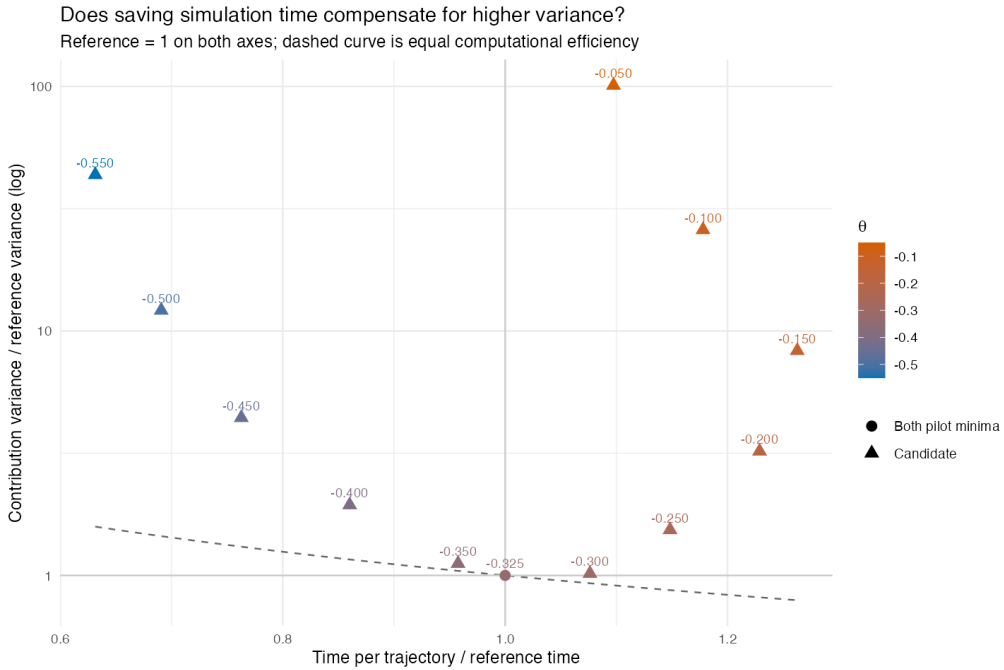
Statistical precision

θ	$\hat{p}$	$10^6\hat{v}$
−0.400	0.0018	6.700
−0.350	0.0018	3.858
−0.325	**0.0018**	**3.448**
−0.300	0.0018	3.512
−0.250	0.0018	5.299

Computational efficiency

θ	Rel. cost	Saved (%)
−0.400	1.671	−67.1
−0.350	1.072	−7.2
−0.325	**1.000**	**0.0**
−0.300	1.096	−9.6
−0.250	1.765	−76.5

Negative savings mean more time for equal precision.



$\frac{Var(\hat{p}_{IS}(\theta_i))}{Var(\hat{p}_{IS}(\theta_j))}$ plotted against $\frac{c(\hat{p}_{IS}(\theta_i))}{c(\hat{p}_{IS}(\theta_j))}$, with fixed $\theta_j = -0.325$

Below the dashed curve, variance times cost is smaller than at θ_j, since

$$\frac{Var(\hat{p}_{IS}(\theta_i))}{Var(\hat{p}_{IS}(\theta_j))}\frac{c(\hat{p}_{IS}(\theta_i))}{c(\hat{p}_{IS}(\theta_j))} = 1 \Rightarrow \frac{Var(\hat{p}_{IS}(\theta_i))}{Var(\hat{p}_{IS}(\theta_j))} = \left(\frac{c(\hat{p}_{IS}(\theta_i))}{c(\hat{p}_{IS}(\theta_j))}\right)$$

Discussion of choices and potential improvements

We have seen how we algorithmically and numerically can reduce computational costs of this importance sampling problem.
However, there remain multiple avenues to explore for further computational easing;

- Iterative benchmarking and profiling for algorithmic optimization
- Moving the time step loops to machine code via `Rcpp` interface
- Implement adaptive importance sampling, and let θ depend on the remaining capital. This could help bridle the tradeoff between ruin probability and likelihood ratio sensitivity.

References I